

Projeto: “FitPlanner” – Sistema de Planejamento Fitness

Descrição do Problema:

Marina decidiu levar uma vida mais saudável e começou a frequentar academia, organizar treinos e cuidar melhor da alimentação. No entanto, ela tem dificuldade em acompanhar sua rotina fitness, controlar os exercícios realizados, registrar metas, monitorar gastos com alimentação e academia, além de manter a disciplina no dia a dia.

Pensando nisso, vamos criar o sistema “**FitPlanner**”, que ajudará Marina (e outros usuários) a organizar sua rotina fitness de forma prática, acompanhando treinos, metas e evolução pessoal.

Requisitos Funcionais:

1. CRUD de Planos de Treino:

O usuário poderá adicionar, visualizar, editar e excluir planos de treino, com informações como: nome do treino, tipo (musculação, cardio, funcional, corrida), data, duração e objetivo.

2. Cadastro de Exercícios:

O usuário poderá cadastrar exercícios relacionados a cada treino (como agachamento, supino, corrida e abdominal), informando séries, repetições, tempo ou distância.

3. Controle de Metas:

Além disso, o sistema deverá permitir registrar metas, como perder peso, ganhar massa muscular ou melhorar o condicionamento físico.

4. Acompanhamento de Evolução:

Ao visualizar o plano fitness, o sistema deverá exibir informações sobre a evolução do usuário, como quantidade de treinos realizados, progresso em relação às metas e frequência semanal.

5. Armazenamento de Dados:

Todos os registros serão salvos em arquivo **.csv** ou **.txt**, garantindo que o histórico de treinos, metas e evolução física esteja sempre acessível.

6. Sugestões Personalizadas:

O sistema poderá sugerir exercícios, divisão semanal, hábitos saudáveis, tempo de descanso e até dicas de alimentação.

7. Funcionalidade Extra:

Sejam criativos.

Requisitos não funcionais:

1. Deve ser feito em Python sem o uso de bibliotecas adicionais.

Utilizar a linha de comando para entrada e saída (interação pelo terminal);

a. Exceções de bibliotecas:

os -> os.system("clear") ou "cls".

datetime

random

Se precisar de outra biblioteca, verifique antes com os professores.

2. O trabalho deve ser feito em grupo.

a. Trabalhos individuais perderão 50% da nota.

3. O código deve estar organizado, portanto, deve conter:

a. Funções para dividir o código de forma lógica e evitar repetições;

b. Tratamento de exceções, para garantir que seu código esteja pronto para tratar casos inesperados.

c. Legibilidade do código, incluindo nomeação de variáveis e funções.

4. Deve ser feito um manual do usuário, explicando como utilizar a ferramenta e restrições gerais que a aplicação tenha.

a. Fiquem à vontade para escolher como será feito esse manual. Pode ser um pdf, site, vídeo, carta...

b. O manual é o readme que pode estar disponível no github do projeto (caso o grupo utilize o github, opcional, não será visto em sala de aula).

5. Não será aceito entregas atrasadas.

6. Apresentação:

a. A equipe deve apresentar o projeto feito para os professores.

b. Todos envolvidos da equipe devem explicar alguma parte, e perguntas direcionadas serão feitas durante a apresentação.

7. A entrega será em uma atividade do classroom

a. O que deve ser entregue: Código da aplicação e Manual do usuário.

Critérios de avaliação:

● Apresentação (50 pontos - nota individual):

○ Participação durante a apresentação do projeto;

○ Perguntas durante a apresentação.

● Código (50 pontos - nota por grupo):

○ Legibilidade e Organização do código;

○ Tratamento de erros e Utilização de Arquivos;

○ Apresentação da ferramenta e manual do usuário;

○ Funcionalidade extra.